

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Developing the System Requirements Specification for Your Own Project

Requirements and System Planning

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 5
Output	System Requirements Specification (SRS), requirements-to-feature traceability matrix, and online requirements review
Maximum Score	80 points

Student / Project Information

Block: <u>A-1</u>	Date: <u>September 20, 2026</u>
System Title: Seaweed (Guso) Quality Classification Using Image Processing	Instructor: Jean Y. Villasin, MIT
Members: Zydane Angelo Adanza, Timothy Josua Tavera, Jherick Peñalosa	

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

1. Activity Alignment and Purpose

Topic Learning Outcome: Analyze the requirements of the group's semester web-system project and produce a scoped requirements specification aligned with the prescribed architecture and selected development methodology.

Course Outcome Alignment: Supports semester-project planning by translating the community problem, user needs, system boundaries, and project constraints into clear and testable requirements that can guide design, implementation, testing, and review.

Syllabus Assessment: System Requirements Specification; requirements-to-feature matrix; online requirements review.

Purpose of the Activity

Activities 2, 3, and 4 established your project architecture, technology stack, and development methodology. In Activity 5, your team will convert the approved community problem and intended user needs into a controlled requirements baseline. You will define functional and non-functional requirements, user roles and permissions, system scope and boundaries, use cases and/or user stories, feature priorities, acceptance criteria, and project constraints. The resulting SRS and traceability matrix will become the reference for the succeeding design, implementation, database, API, testing, and review activities.

Important distinction: A requirement states what the system must do or what quality/constraint it must satisfy. A feature groups related capabilities that users can recognize. Scope defines what the project will and will not deliver. Architecture describes how the solution is organized. Do not replace a requirement with a technology name unless the technology itself is a prescribed project constraint.

2. Intended Learning Objectives

- Analyze the community or organizational problem, target users, stakeholders, and desired system outcomes.
- Write clear, atomic, and testable functional requirements using project-specific actions and data.
- Specify measurable non-functional requirements for security, performance, usability, reliability, compatibility, maintainability, and other relevant quality needs.
- Define user roles, permissions, data-access boundaries, and responsibilities using the principle of least privilege.
- Establish the system scope, out-of-scope items, assumptions, dependencies, and project constraints. • Refine use cases and/or user stories, feature priorities, business rules, and acceptance criteria based on the selected development methodology.
- Assemble a coherent SRS and requirements-to-feature traceability matrix aligned with Flutter Web, the REST API, PostgreSQL, and the team's selected methodology, then prepare the requirements baseline for online review.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to requirements analysis, refinement, traceability, and review.

Activity Item	Requirement
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled semester-project consultation and online requirements review.
Primary Output	One project-specific System Requirements Specification (SRS) and one Requirements-to-Feature Traceability Matrix.
Supporting Output	Problem/user analysis, functional and non-functional requirements worksheets, role-permission matrix, scope and boundary worksheet, use-case/user-story refinement, feature-priority and acceptance-criteria plan, and review notes.
Recommended Tools	Word/Google Docs, diagrams.net/Draw.io or equivalent for use-case diagrams, spreadsheet/table tool for traceability, Git/GitHub, and the team's current project board or documentation workspace.
Review Format	Online requirements review through the course platform, with instructor consultation or revision requests when requirements are incomplete, ambiguous, out of scope, or inconsistent with the prescribed architecture.

4. Pre-Laboratory Requirements

- Initial list of target users, stakeholders, major features, data to be collected, and any external services or integrations.
- Available requirement evidence such as stakeholder interview notes, observation results, existing forms/processes, instructor feedback, policy/rule references, or project consultations.
- Access to the project GitHub repository or shared documentation workspace where the SRS, diagrams, review notes, and traceability matrix will be maintained.

5. Part A - Community Problem and Target-User Analysis

Begin with the actual problem before writing features. Requirements should be traceable to a real user need, stakeholder rule, project constraint, or system-quality objective. Complete the project-context worksheet and the target-user/stakeholder analysis before defining detailed requirements.

Project Context Item	Your Project Evidence / Statement
Official system title	Guso Quality Classification Using Image Processing
Community / organizational problem	Farmers and sellers need a more consistent and convenient way to assess Guso quality. Manual visual grading can vary between people and may be difficult to document consistently.
Current process or problem evidence	Project problem analysis and team/adviser consultations identified manual image-based quality assessment and the need for a repeatable digital classification workflow. Replace/append this statement with actual interview, observation, or consultation evidence if available.
Target community / organization	Guso/seaweed farmers and sellers who need to assess and keep records of Guso quality.

Primary users / beneficiaries	Farmers and Sellers
Secondary users / stakeholders	Project adviser/instructor and the student development team; future evaluators or authorized project stakeholders may review system outputs.
Desired system outcome and success indicator	Users can register and verify an account, upload a valid Guso image, receive a MobileNetV3-based quality classification result, and view/save the resulting record in the system history.

Target User and Stakeholder Analysis Worksheet

User / Stakeholder	Main Goal or Problem	Expected System Interaction	Requirement Source / Evidence
Role 1	Farmer	Assess Guso quality consistently and keep classification records.	Register/verify, log in, upload Guso image, classify, view result, view history, download/save record.

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 3 of 12

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

User / Stakeholder	Main Goal or Problem	Expected System Interaction	Requirement Source / Evidence
Role 2	Seller	Check Guso quality before selling or for record keeping.	Register/verify, log in, upload image, classify, view result, view history, download/save record.
Role 3	Adviser / Instructor	Check whether the system requirements and implementation are aligned with the project.	Review demonstrations, provide feedback, confirm scope and requirements.
Role 4	Development Team	Build, test, integrate, and maintain the semester project.	Implement Flutter Web, FastAPI, PostgreSQL, authentication, ML integration, testing and deployment.
Role 5			
Role 6			
External stakeholder 1			
External stakeholder 2			
Other			
Other			

6. Part B - Functional and Non-Functional Requirements

Write requirements so another team member can implement and test them without guessing. Give each requirement a unique ID. Functional requirements describe system behavior or services. Non-functional requirements describe measurable quality expectations or constraints. Use actual users, data, rules, and project conditions rather than generic statements.

Requirement ID / Type	Requirement Statement	Source / Rationale	Priority	Verification / Acceptance Evidence
FR-01	Functional	The system shall allow a Farmer or Seller to register an account using a contact number and password.	Account creation	Must
FR-02	Functional	The system shall send an OTP verification step for a newly registered account and mark the account verified only after a valid, unexpired OTP is confirmed.	Authentication / security	Must
FR-03	Functional	The system shall allow a verified user to log in and access authenticated functions using valid credentials and an authentication token.	Authentication	Must
FR-04	Functional	The system shall allow an authenticated Farmer or Seller to upload a supported Guso image for classification.	Guso classification	Must
FR-05	Functional	The FastAPI backend shall validate and preprocess the uploaded image before passing it to the integrated MobileNetV3 model.	Classification workflow	Must
FR-06	Functional	The system shall return a Guso quality classification result and confidence score to the authenticated user.	Automated quality grading	Must
FR-07	Functional	The system shall save the classification result, timestamp, user reference, and related Guso image record in PostgreSQL.	History / records	Must
FR-08	Functional	The system shall allow an authenticated user to view their own classification history and download/save a classification record.	History / download	Must
FR-09	Functional	The system shall display a responsive dashboard containing the user's relevant classification activity and record access.	Dashboard	Should
FR-10	Functional	The system shall display clear validation and error messages for invalid images, failed authentication, unavailable API/database services, and failed classification requests.	Reliability / usability	Must

NFR-01	Security	Passwords shall be stored as secure password hashes, not plaintext; OTP values shall be protected and expire.	Must	NFR-01
NFR-02	Security	All client-to-API communication shall use HTTPS in deployed environments.	Must	NFR-02
NFR-03	Security	FastAPI shall enforce authentication/authorization and server-side validation; Flutter validation shall not be the only security control.	Must	NFR-03
NFR-04	Performance	Under normal project test conditions, a valid classification request should return a result within a team-defined target (recommended initial target: 10 seconds excluding unusually slow network conditions).	Should	NFR-04
NFR-05	Usability	The Flutter Web interface shall provide readable feedback for loading, successful classification, invalid input, and service failure states.	Must	NFR-05
NFR-06	Reliability	A failed classification or database save shall not be presented as a successfully saved record.	Must	NFR-06
NFR-07	Compatibility	The web client shall support current mainstream desktop/mobile browsers targeted by the team and use responsive layouts for the intended devices.	Should	NFR-07
NFR-08	Maintainability	Flutter client, FastAPI backend, database scripts/migrations, and ML integration shall be separated into maintainable modules and tracked in Git/GitHub.	Should	NFR-08
NFR-09	Data integrity	PostgreSQL shall enforce primary keys, foreign keys, uniqueness for contact numbers, and appropriate constraints for classification records.	Must	NFR-09
NFR-10	Deployment	Production secrets such as database credentials, token secrets, and OTP	Must	NFR-10

		provider credentials shall be stored in environment configuration and not committed to the repository.		
NFR-08	Maintainability	Flutter client, FastAPI backend, database scripts/migrations, and ML integration shall be separated into maintainable modules and tracked in Git/GitHub.	Should	NFR-08

Requirement-writing rule

Prefer a testable form such as “The system shall ...”. Keep one requirement per statement. Replace vague words such as fast, secure, easy, user-friendly, reliable, or real-time with measurable conditions whenever the project can define them. A functional requirement should identify the actor or trigger, required behavior, important data or rule, and expected outcome. A non-functional requirement should state the quality attribute, condition, target or threshold, and how it can be verified.

7. Part C - User Roles and Permissions

Define the actual access level of every system role. The role-permission matrix must agree with the use cases/user stories and with the authentication and authorization controls shown in the architecture. Do not give every user full access simply to simplify implementation.

User Role	Purpose / Main Tasks	Allowed Actions	Restricted /	Data Access Scope	Authentication / Permission Rule
Role 1	Farmer	Assess Guso quality and manage own records.	Register, OTP verify, login, upload image, classify, view own result/history, download/save own records.	Cannot access another user's records or administrative controls.	Own account, own uploaded images, own classification records.
Role 2	Seller	Assess Guso quality and manage own records.	Register, OTP verify, login, upload image, classify, view own result/history, download/save own records.	Cannot access another user's records or administrative controls.	Own account, own uploaded images, own classification records.
Role 3	Adviser / Instructor	Review project progress and requirements.	Review demonstrations, provide feedback, inspect submitted project artifacts.	No automatic access to private user records unless explicitly authorized by the team/instructor and implemented.	Project artifacts; no default end-user data access.
Role 4					

Role 5					
External / limited role					

Role and access review checks:

- Assign only the permissions required for each role to complete its responsibilities.
- Identify which records a user may view, create, edit, approve, delete, export, or administer.
- Separate sensitive or administrative actions from ordinary user actions and define where authorization must be enforced.
- Ensure each permission is reflected in at least one requirement, use case/user story, or explicit out-of-scope rule.

8. Part D - System Scope, Boundaries, Assumptions, and Constraints

Selected methodology from Activity 4: Scrum (iterative and incremental delivery).

Prescribed architecture: Flutter Web → REST API → PostgreSQL. The same FastAPI backend can serve the cross-platform Flutter mobile client if the mobile client is included in the semester release.

System objective / intended outcome (2-3 sentences):

The Guso Quality Classification system will provide Farmers and Sellers with a web-based interface for submitting Guso images, receiving automated quality classifications through an integrated MobileNetV3 model, and maintaining accessible classification records. The system is intended to make the assessment workflow more consistent, traceable, and convenient while remaining feasible within the semester project scope.

Scope and Boundary Worksheet

For each major module or feature area, state what the semester project will deliver, what will remain outside the project boundary, who uses it, what data it needs, and which dependencies or constraints affect implementation.

Module / Area	In Scope	Out of Scope	User(s)	Main Data	Dependency / External Service	Constraint / Rationale
1. Authentication	Registration, password-based login, OTP verification, authenticated sessions/tokens.	Social login, biometric authentication, advanced identity verification.	Farmers, Sellers	Users, OTP verification records	OTP/SMS provider if SMS delivery is implemented	Semester scope and team capacity.
2. Guso Classification	Image upload, client/server validation, preprocessing, MobileNetV3 inference, quality result + confidence.	Training a new production-scale model, automated grading for unrelated crops/products.	Farmers, Sellers	Guso images, classification records	MobileNetV3 integrated in FastAPI	ML data/model quality and compute limits.
3. History / Records	Store and retrieve user's classification history and timestamps.	Enterprise analytics, public marketplace records, cross-user data sharing.	Farmers, Sellers	Classification records/images	PostgreSQL	Privacy and storage limits.

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

Module / Area	In Scope	Out of Scope	User(s)	Main Data	Dependency / External Service	Constraint / Rationale
4. Dashboard	Responsive dashboard showing relevant user activity and access to records.	Advanced business intelligence dashboards.	Farmers, Sellers	User activity / classification summaries	PostgreSQL via FastAPI	Semester time and UI scope.
5. Download / Save Record	Allow users to save/download their own classification record.	Public sharing marketplace, automated cloud document management.	Farmers, Sellers	Classification result data	FastAPI + Flutter Web	Export format kept simple for semester scope.
6. Administration	Basic backend role/authorization controls needed to protect data and endpoints.	Full enterprise admin console, billing, organization management.	System/backend team	Users/authorization data	FastAPI + PostgreSQL	Only implement admin functions that are actually approved.
7						
8						

Boundary rule

“In scope” means the team commits to analyze, implement, test, and demonstrate the item within the semester. “Out of scope” means it is intentionally excluded from the current version, even if it may be useful in the future. Assumptions are conditions believed to be true; dependencies are external items the system relies on; constraints are limits the team must work within. All four should be explicit to prevent uncontrolled scope growth.

9. Part E - System Use Case Diagram and Use-Case/User-Story Refinement

Model the required behavior from the user's point of view. Every group must first create one complete UML use case diagram for its own approved semester system. The diagram must be based on the actual user roles, functional requirements, scope, and permissions defined in this activity. After the diagram is completed, refine the major use cases and/or user stories in the worksheet below. Teams using a predictive approach may emphasize detailed use-case descriptions, while teams using an adaptive/Agile approach may also maintain user stories and acceptance criteria. The diagram and the written requirements must remain consistent.

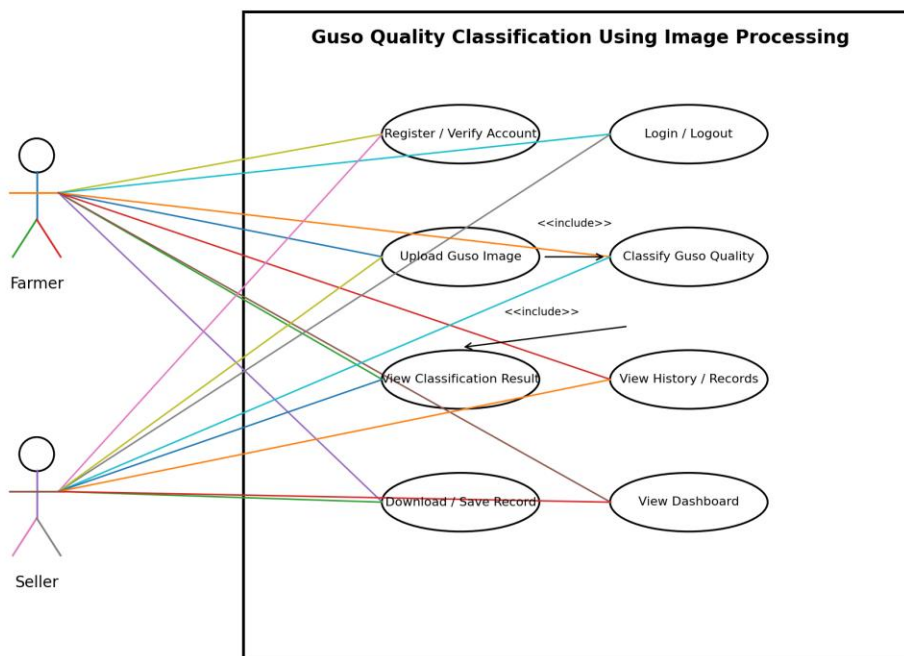
Mandatory Use Case Diagram Requirement

Each group must create a UML use case diagram for its own approved system. The diagram is a required Activity 5 output and must be inserted in the SRS or attached as a clearly labeled figure.

- Draw one system boundary rectangle labeled with the official system title.
- Place all major actors or user roles outside the system boundary.
- Show the major user-goal use cases inside the boundary using clear verb-based names (for example:

Submit Report, Approve Request, View Map, or Manage Account).

- Connect actors to the use cases they are allowed to perform. Use <<include>> and <<extend>> only when they are logically justified.
- Ensure all Must/High-priority functional features are represented and that the diagram agrees with the role-permission matrix, system scope, and functional requirements.
- Do not place Flutter, the REST API,s PostgreSQL, database tables, screens, or other internal implementation components as actors or use cases. These belong to architecture/design artifacts, not the UML use case diagram.
- Required files: readable PNGof the final use case diagram plus the editable source file (for example .drawio, .fig, or equivalent).



ID	Actor / User	Goal / Need	Trigger / Main Interaction	Expected Result / Acceptance
UC/US-01	Farmer/Seller	Register and verify account	User submits contact number/password and completes OTP verification.	Verified account is created and can authenticate.
UC/US-02	Farmer/Seller	Log in	Verified user submits valid credentials.	Authenticated session/token is established; invalid credentials are rejected.
UC/US-03	Farmer/Seller	Upload Guso image	Authenticated user selects a supported image and submits it.	Valid image is accepted; invalid image shows a clear error.
UC/US-04	Farmer/Seller	Classify Guso quality	User submits a valid image for classification.	FastAPI validates/preprocesses it, MobileNetV3 produces a

				result, and the result is returned.
UC/US-05	Farmer/Seller	View classification result	Classification completes successfully.	Quality grade and confidence are displayed.
UC/US-06	Farmer/Seller	View history / records	Authenticated user opens history.	Only the user's classification records are displayed.
UC/US-07	Farmer/Seller	Download / save record	User selects an owned record.	The selected classification record can be saved/downloaded.

Refinement rule

For a use case, identify the actor, goal, precondition when relevant, main flow, alternative/error flow, and expected postcondition. For a user story, use a form such as “As a [role], I want [capability], so that [value]” and attach acceptance criteria. Every Must/High priority functional requirement should link to at least one use case, user story, or clearly defined system event.

10. Part F - Feature Priorities, Acceptance Criteria, and Project Constraints

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Feature prioritization	Choose and apply one consistent priority scheme such as Must/Should/Could/Won't-for-this-semester or High/Medium/Low.	Must: authentication/OTP, image upload + validation, MobileNetV3 classification, classification result, history/records. Should: responsive dashboard, download/save record. Could: additional analytics or advanced export if time permits. Won't-for-this-semester: unrelated enterprise features.
Acceptance criteria	Define observable conditions that must be true before a requirement or feature is considered complete.	A feature is accepted only when its defined functional behavior works, error cases are handled, required data is stored/retrieved correctly, and evidence is available through testing/demo.
Business rules	Identify approval rules, calculations, validations, status transitions, eligibility rules, or other domain logic.	Only authenticated users can classify images or access records. A user may access only records belonging to that user. Invalid/corrupt images must not be passed to ML inference. A classification is not marked saved until the database operation succeeds.
Project constraints	Record deadline, team size, required stack, hardware/device, hosting, data-access, policy, budget, or other fixed limits.	Semester deadline; small student team; prescribed Flutter Web → REST API → PostgreSQL architecture; MobileNetV3 integration; limited development hardware/time; selected hosting platforms.
Data and privacy constraints	State which data are sensitive, who may access them, retention/sharing limits, and minimum protection needed.	Contact numbers, password hashes, OTP verification records, uploaded Guso images, and classification history are protected data. Passwords and OTPs

		are not stored in plaintext. User records are access-controlled.
External-service dependencies	Identify maps/GIS, ML service/model, email/SMS, weather, storage, or other services and what happens when they are unavailable.	OTP/SMS provider if used; hosting availability; Internet connectivity. MobileNetV3 is integrated into FastAPI rather than a separate external ML API.
Performance / reliability targets	Define realistic response-time, availability, data consistency, recovery, or workload expectations when relevant.	Initial proposed target: valid classification result within 10 seconds under normal test conditions, excluding unusually slow network conditions. Failed saves must not be reported as successful.
Validation and error handling	Define required input validation, user feedback, failed request handling, and recovery behavior.	Validate image type, size, and corruption on client and server; verify authentication server-side; return meaningful HTTP status/error responses; show retry guidance for transient failures.

Recommended minimum requirement evidence in the project repository or shared workspace:

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 7 of 12

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

```
docs/requirements/
  system-requirements-specification.docx or .pdf
  requirements-to-feature-matrix.xlsx or .csv
  use-cases-or-user-stories/
  role-permission-matrix/
  scope-and-constraints/
  review-notes/
README.md - current requirements baseline/version and status
```

11. Part G - Construct the System Requirements Specification

Assemble the completed analysis into one coherent SRS. The document must be project-specific and internally consistent. Do not copy generic requirements from another system. Use stable requirement IDs so changes can be traced during implementation and testing.

No.	Required SRS Element	What Must Be Shown
1	Document control and project identity	System: Guso Quality Classification Using Image Processing. Baseline: 1.0. Methodology: Scrum. Architecture: Flutter Web → REST API → PostgreSQL. ML model: MobileNetV3 integrated into FastAPI.
2	Problem and target users	The project addresses the need for a more consistent, convenient, and recordable Guso quality assessment workflow. Primary users are Farmers and Sellers.
3	System objectives	Provide verified user accounts; accept valid Guso images; classify image quality through MobileNetV3; display classification and confidence; store history; and allow users to retrieve/save their own records.
4	Scope and boundaries	The current semester scope covers authentication/OTP, Guso image classification, history/records, dashboard, and download/save record. Advanced enterprise administration, unrelated crop classification, and large-scale commercial analytics are

		outside scope.
5	User roles and permissions	Farmers and Sellers have equivalent application permissions and can access only their own records. Adviser/instructor is a project stakeholder rather than a default end-user role.
6	Functional requirements	FR-01 through FR-10 define registration, OTP verification, login/token use, image upload, validation/preprocessing, MobileNetV3 classification, persistence, history, download/save, dashboard, and error handling.
7	Non-functional requirements	Security, HTTPS, server-side authorization/validation, performance, usability, reliability, compatibility, maintainability, data integrity, and deployment-secret protection are defined as NFR-01 through NFR-10.
8	Use cases and/or user stories	UC/US-01 through UC/US-08 cover registration/verification, login, upload, classification, result viewing, history, download/save, and dashboard.
9	Features and priorities	Must features are authentication/OTP, upload/validation, classification, result, and history. Dashboard and download/save are planned as Should features, with prioritization adjusted through Scrum reviews.
10	Acceptance criteria and business rules	Observable acceptance conditions are defined in the use-case and requirement tables. Core rules include ownership, authentication, server-side validation, and successful database confirmation before marking a record saved.
11	Architecture and data alignment	Flutter Web/Riverpod handles UI, navigation, local state, and client validation. FastAPI handles routes, authentication/authorization, validation, business/classification logic, image processing, MobileNetV3 inference, and response generation. PostgreSQL stores users, OTP verification records, classification records, and classification images.
12	Traceability and review status	FR/NFR IDs are mapped to features/use cases, architecture/data responsibilities, and verification evidence in Part H. The baseline should be updated after adviser/instructor review.

Minimum SRS Rule

Every Must/High-priority feature must be supported by one or more functional requirements and any relevant non-functional requirements. Each requirement must have a unique ID, a clear source or rationale, a priority, and a verification/acceptance method. Requirements must remain feasible within the selected methodology, semester scope, and prescribed Flutter Web -> REST API -> PostgreSQL architecture.

12. Reference Example - Do Not Copy as Your Final Output

The example below illustrates the analysis and traceability logic expected in Activity 5. Replace every generic label with the actual community problem, users, requirements, features, roles, constraints, architecture components, and acceptance evidence of your own semester project.

Reference Requirements Analysis and Traceability Flow

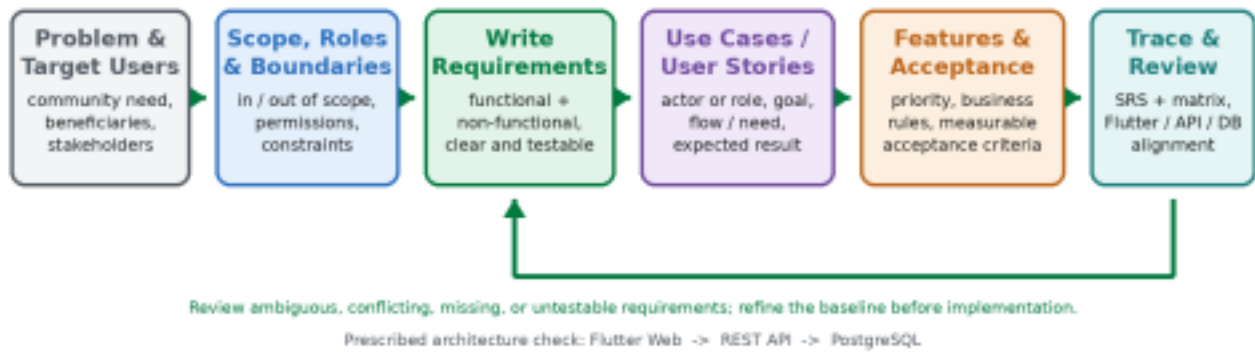


Figure 1. Reference requirements analysis and traceability flow.

Step	Reference Flow
1	Start from the verified community/organizational problem, target users, stakeholders, and requirement evidence.
2	Define system scope, role permissions, boundaries, assumptions, dependencies, and project constraints.
3	Write uniquely identified functional and non-functional requirements that are clear, feasible, and testable.
4	Refine the behavior through use cases and/or user stories that reflect the team's selected development methodology.
5	Group requirements into prioritized features and define measurable acceptance criteria and business rules.
6	Trace requirements to features, Flutter Web/API/PostgreSQL responsibilities, acceptance evidence, and review status; revise gaps before implementation.

13. Part H - Requirements-to-Feature Traceability Matrix

System module / feature set:

Requirements baseline version / date:

Requirement ID	Requirement Summary	Linked Feature & Use Case	Stack Component / Data	Acceptance / Verification & Status
FR-01	Account registration	UC/US-01	Flutter Web + FastAPI + users	Successful account creation with unique contact number
FR-02	OTP verification	UC/US-01	FastAPI + otp_verifications + OTP provider if used	Valid/expired/incorrect OTP tests
FR-03	Login/authentication	UC/US-02	Flutter Web + FastAPI authentication/token verification	Valid/invalid credential and token tests
FR-04	Guso image upload	UC/US-03	Flutter Web + FastAPI + classification_images	Supported/unsupported image tests
FR-05	Server validation + preprocessing	UC/US-04	FastAPI + MobileNetV3 pipeline	Corrupt/invalid/valid image test evidence
FR-06	Quality classification result	UC/US-04/05	FastAPI + MobileNetV3	Classification output + confidence displayed
FR-07	Save classification record	UC/US-05/06	FastAPI + PostgreSQL classification_records/images	Database insert and retrieval evidence
FR-08	History + download/save	UC/US-06/07	Flutter Web + FastAPI + PostgreSQL	Ownership and record export/retrieval test
FR-09	Responsive dashboard	UC/US-08	Flutter Web + Riverpod + FastAPI/PostgreSQL	Responsive UI test
FR-10	Error handling	UC/US-02/03/04/06	Flutter Web + FastAPI	Failure scenario tests

14. Part I - Requirements Review and Baseline Approval

Before submission, conduct a structured group review and prepare the SRS for the online requirements review. Record issues that must be revised instead of silently changing the baseline. Confirm all of the following:

- Every requirement is necessary, understandable, uniquely identified, feasible within the semester, and testable or verifiable.
- Functional requirements, non-functional requirements, roles/permissions, scope, use cases/user stories, features, priorities, and acceptance criteria agree with one another.
- No Must/High-priority feature lacks supporting requirements or acceptance evidence, and no requirement is orphaned from a feature, user need, or system-quality objective.
- The requirements are consistent with the selected development methodology and with the prescribed Flutter Web REST API-PostgreSQL architecture.
- Ambiguous, conflicting, duplicate, missing, or out-of-scope requirements are identified and assigned a revision decision before implementation begins.

Online review status: ☐ Ready for review ☐ Requires revision ☐ Reviewed with comments Reviewer / consultation source: _____ Date: _____

_____ Baseline version: _____ Most important revision identified during review: _____

15. Final Quality Checklist

Checklist A	Checklist B
☐ The community/organizational problem and target users are clearly stated and supported by project evidence.	☐ Every requirement has a unique ID and uses project-specific, understandable wording.
☐ Functional requirements describe required behavior rather than vague features or implementation details.	☐ Non-functional requirements are measurable or verifiable where practical.
☐ User roles, permissions, restricted actions, and data-access boundaries are consistent across the SRS.	☐ In-scope and out-of-scope items are explicit; assumptions, dependencies, and constraints are documented.

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 10 of 12

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

Checklist A	Checklist B
☐ Use cases and/or user stories are linked to actual roles and required system behavior.	☐ Features are prioritized consistently and acceptance criteria are observable and testable.
☐ Business rules, validation, error handling, and external-service dependencies are documented when relevant.	☐ Requirements are aligned with Flutter Web, the REST API, PostgreSQL, and approved external services.
☐ The requirements-to-feature matrix contains no unexplained orphan requirements or unsupported Must/High features.	☐ The SRS and traceability matrix agree with the selected development methodology and semester plan.
☐ The requirements baseline has been reviewed for ambiguity, conflict, duplication, feasibility, and scope creep.	☐ Every group member can explain assigned requirements, role permissions, acceptance criteria, and traceability decisions during consultation or review.

16. Submission Requirements

- Final System Requirements Specification in editable DOCX and PDF format, using the team's current requirements baseline/version.
- Completed Requirements-to-Feature Traceability Matrix in the SRS or as an accompanying spreadsheet/table file. • Completed role-permission matrix, scope/boundary worksheet, use-case diagram and/or user-story set, feature-priority list, and acceptance-criteria evidence.
- Updated project GitHub repository or shared workspace containing the requirement artifacts under a folder such as docs/requirements/.
- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA5_SystemName.
- Complete the online requirements review. If the instructor returns comments or identifies requirement defects, submit the revised baseline and preserve a brief record of the changes made.

Academic Integrity and Competency Validation

The SRS must represent the team's own approved project, stakeholder evidence, scope decisions, and technical constraints. AI tools may be used to clarify terminology, refine wording, or check consistency, but generated requirements must not be accepted without verification against the team's actual system. Every member is responsible for understanding the requirements assigned to the group. The instructor may ask any member to explain the source, purpose, priority, permission rule, acceptance criterion, or implementation layer of a requirement without assistance.

17. Scoring Rubric - 80 Points

Partial credit may be awarded when evidence is present but incomplete, ambiguous, inconsistent, untestable, outside the approved scope, or not traceable to the team's actual project. Requirements returned during the online review should be revised before they are used as an implementation baseline.

Criterion	Evidence for Full Credit	Points
Problem, target users, and stakeholder analysis	The SRS clearly identifies the real community/organizational problem, current situation, target users, stakeholders, goals, and evidence used to derive requirements.	8
Functional requirements	Functional requirements are complete for the approved scope, uniquely identified, project specific, atomic, feasible, clear, prioritized, and testable.	14
Non-functional requirements	Relevant quality requirements are measurable/verifiable and appropriately address security, performance, usability, reliability, compatibility, maintainability, deployment, or other project needs.	10

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

Criterion	Evidence for Full Credit	Points
User roles, permissions, and scope boundaries	Roles, allowed/restricted actions, data access, in-scope/out-of-scope items, assumptions, dependencies, and constraints are accurate and mutually consistent.	10
Use cases/user stories and feature priorities	Use cases and/or user stories correctly represent user goals and required behavior; features are logically grouped, linked to requirements, and prioritized for the semester.	10
Acceptance criteria, business rules, and constraints	Acceptance conditions are observable/testable; validation, calculations, approvals, errors, external dependencies, and important project constraints are documented when relevant.	8
Requirements-to-feature traceability and architecture alignment	The matrix links requirements to features/use cases or stories, Flutter Web/API/PostgreSQL responsibilities or data, and acceptance evidence without unexplained gaps or contradictions.	12
SRS quality and online requirements review	The document is organized, readable, versioned, internally consistent, submitted correctly, and reviewed for ambiguity, duplication, conflict, feasibility, traceability, and scope before implementation.	8

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which requirement was the most difficult to make clear and testable, and what did your group change?

The most difficult requirement was the Guso classification requirement because 'real-time' or 'accurate' can be vague. We changed it into observable steps: the system must validate the image, process it through MobileNetV3, return a classification and confidence score, and record the result after a successful database save. A measurable response-time target can be validated during testing.

Which requested feature or capability was placed out of scope for this semester, and why?

Advanced analytics, enterprise administration, public sharing/marketplace functions, and large-scale multi-crop classification are out of scope because the semester project must prioritize authentication, Guso classification, history/records, dashboard, and basic record saving within the available time and team capacity.

Which user role has the most security-sensitive permissions, and what control should protect those actions?

The ordinary Farmer and Seller roles have access to their own sensitive account and classification data, so the most important control is server-side authentication and ownership authorization in FastAPI. No user should retrieve another user's records by changing an ID in an API request.

Which non-functional requirement has the greatest effect on your Flutter Web-REST API-PostgreSQL architecture?

Security has the strongest architectural effect because HTTPS, password hashing, OTP protection, token verification, server-side validation, authorization, and secret management must be enforced across the client, API, database, and deployment environment.

What requirement changed or became clearer after the requirements review, and how will your selected methodology manage that change?

The exact classification workflow and validation rules may become clearer after prototype and model testing. Scrum will record the change as a backlog item, assess its effect on scope/time/risk, and prioritize it for the next sprint or seek adviser approval if it affects a committed milestone.

